

Executive Summary Report

NewsBot Intelligence System 2.0

A Business-Focused Overview of My Final Project

Student: Nertnapha Sanwiwat

Course: ITAI 2373 — Natural Language Processing

Institution: Houston Community College

Project Type: Individual Project

Project Overview

For this project, I wanted to solve a problem that real newsrooms and analysts actually deal with: there is way too much news being published every day (over a million articles worldwide) across too many languages for anyone to read and organize by hand. NewsBot Intelligence System 2.0 is my attempt at a single tool that can classify, summarize, translate, and even answer questions about the news automatically, instead of relying on someone doing all of that manually.

The system has four main parts working together: a Content Analysis Engine (classification, topic modeling, sentiment tracking, entity mapping), a Language Understanding & Generation layer (summarization, semantic search, content enhancement), a Multilingual Intelligence layer (language detection, translation, cross-lingual comparison), and a Conversational Interface (so a user can just type a question instead of digging through a dashboard). I also built the whole thing to keep working even if one piece fails, so a single broken component doesn't take down the entire system.

85%+ Classification accuracy across 5 news categories	4 Integrated NLP modules in one system	3+ Languages supported (EN / ES / FR)
---	--	---

Value Proposition

The main idea behind NewsBot 2.0 is that you don't need an expensive enterprise media-monitoring subscription to get useful, automated news intelligence. Because it's built mostly on open-source tools and pretrained models, it's a lighter-weight option, and since I built every piece of it myself, I know exactly how each result is generated instead of trusting a black box.

For Newsrooms

- Automates tagging and summarization, so editors can spend more time on original reporting instead of manual sorting.

For Analysts & Researchers

- Tracks sentiment and topic trends across thousands of articles in minutes instead of days of manual review.

For Organizations Working Across Languages

- Compares how the same story is being covered in different languages, which would normally require several bilingual analysts.

Return on Investment (ROI) Analysis

The main way NewsBot 2.0 would save time and money is by cutting down the hours people currently spend reading, tagging, and comparing articles by hand. Since almost the whole system runs on open-source libraries and pretrained models (spaCy, scikit-learn, Hugging Face Transformers), the biggest cost isn't licensing fees — it's the time it takes to build, run, and maintain it, which is very different from most commercial media-monitoring tools that charge per seat or per query.

Cost / Benefit	Doing It Manually	Using NewsBot 2.0
Sorting and tagging articles	Manual reading and categorizing, hours per day per person	Automatic classification with a confidence score in seconds
Tracking trends and sentiment	Occasional manual review, insights arrive late	Continuous, automatic tracking
Comparing coverage across languages	Needs staff who speak multiple languages	Automatic detection, translation, and comparison
Cost structure	Per-seat or per-query subscription for commercial tools	Mostly open-source; main cost is compute and dev time

Because the models are already pretrained (BART for summarization, DistilBERT for sentiment, MiniLM for semantic search), scaling up to handle more articles mostly costs compute time, not new licensing fees.

Competitive Analysis

I compared NewsBot 2.0 to two alternatives: large commercial media-monitoring platforms, and doing everything manually. The biggest advantages of my system are that it's transparent (I can see exactly how it works, since I built it) and that it doesn't cost much beyond compute time.

Dimension	Commercial Media-Monitoring Tools	Doing It Manually	NewsBot 2.0
Cost	High, recurring fees per seat/query	Low direct cost, high labor cost	Low recurring cost, open-source
Transparency	Usually a black box	Fully transparent (a human decides)	Transparent, I can see every step
Multilingual support	Varies, often an added cost	Limited by staff language skills	Built in (EN / ES / FR)
Conversational access	Rare, usually just a dashboard	Not applicable	Built-in natural-language interface
Room to grow	Locked to vendor's roadmap	Not applicable	Modular, easy to extend

The biggest gap right now compared to mature commercial tools is that NewsBot 2.0 hasn't been tested at large scale yet, and it only supports three languages so far. Both of those are addressed in the roadmap below.

Current Results

- Classification accuracy is above 85% across the five news categories I tested.
- Sentiment tracking lines up closely with the human-labeled samples I checked it against.
- Topic coherence is strong for single-theme categories like business and politics, and weaker on short, mixed-topic articles — an area I'd want to improve.
- The summarizer is tuned to sound fluent and readable; making sure it captures every important fact is the next thing to work on.

Roadmap: NewsBot 3.0

What I'd Add Next	Why It Matters
Fine-tuned transformers (BERT/roBERTa trained on news)	Should push classification accuracy even higher
Real-time streaming	Get insights as news breaks instead of waiting for a batch run
Bias and fact-check detection	Would make the system's insights more trustworthy
A web app front end	Would let non-technical users try it without opening a notebook

Conclusion

This project showed me that a modular system built mostly on free, pretrained models can still do a lot of the same things as expensive commercial media-monitoring tools — classification, sentiment tracking, entity mapping, summarization, multilingual comparison, and even a conversational interface — without the licensing costs. If I kept working on this, my next priority would be real-time streaming and a simple web front end, since those two changes would take it from a class project to something people could actually use day to day.